Input : $n \geq 2$ points $p_1(x_1, y_1)$, $p_2(x_2, y_2)$, . . . , $p_2(x_n, y_n)$
Output: p_i p_j such that $d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (x_i - y_j)^2}$

```
class Point:
     x: float
     y: float

def brute_force(points)
     d = inf
     for i in  points
          for j in points[i:]:
               d = min(d,dist(pi,pj))

def closest_pair(px: list,py: list) -> (Point,Point): #px sorted by x-coord and py
y-coord
     if n <= 3:
          return #Bruteforce

     lx = px[:len(px)//2]
     ly = # Elements of lx sorted on y

     rx = px[len(px)//2:]
     ry = # Elements of rx sorted on y

     l = (l1,l2) = closest_pair(lx,ly)
     r = (r1,r2) = closest_pair(rx,ry)
     d = dist (l,r)
     (s1,s2) = closest_split_pair(px,py,d)
     return best of em

 def closest_split_pair(px,py,d):
      X = largest x coord in the left half
      Sy = {q_1, q_2,... q_l} with x coord restricted to [X-d,X+d]
      closest_distance = d
      closest_pair = null

      for i, __ in enum(Sy[:-1]):
           for j in  range(i+1,min(l,i+7)):
                if (d(q_i,q_j) < closest_distance):
                     closest_distance = d(q_i,q_j)
                     cloasest_pair = (q_i,q_j)
```